

COSMOS Cluster 12

Machine Learning

Summer 2026

Topic: Natural Language Processing

Instructors: Sam Lau and Sooh Liao

Teacher Fellow: Emmanuel Pizano

UC San Diego

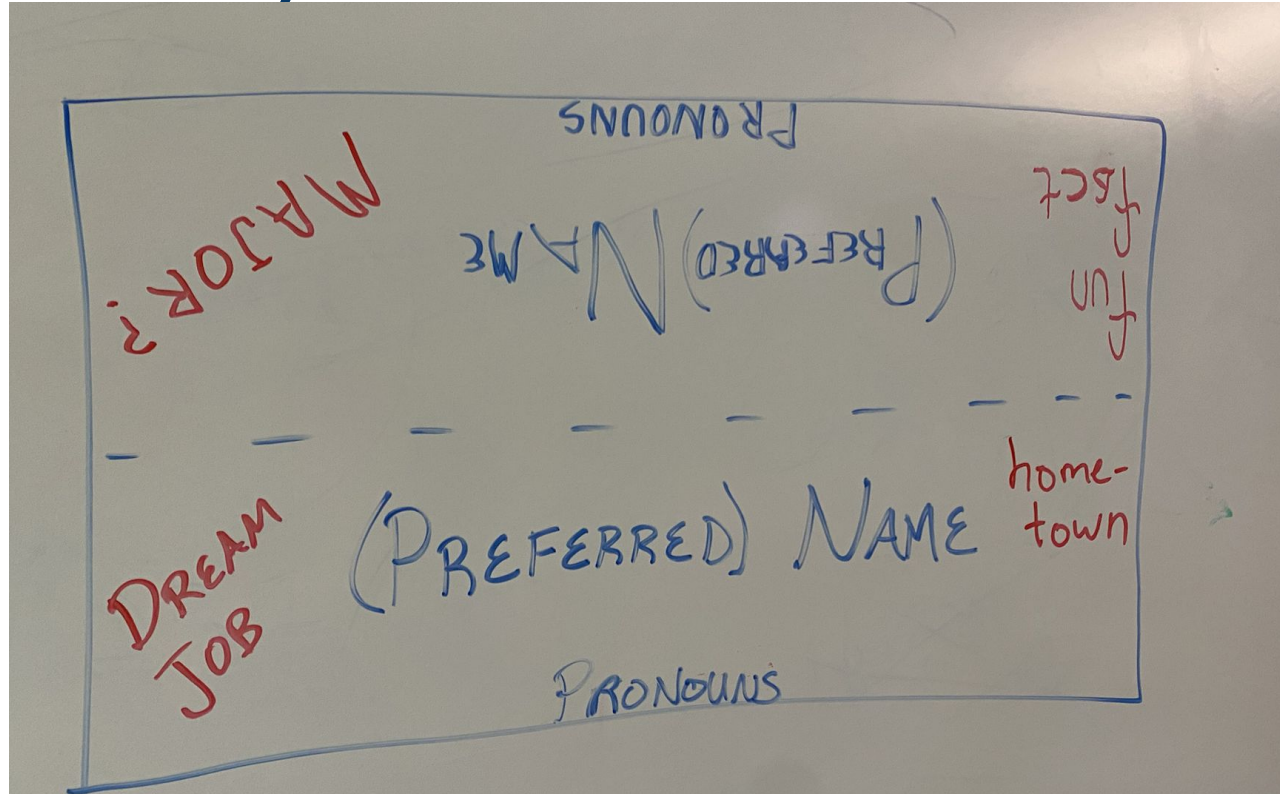


Sooh Nam Liao

Assistant Teaching Professor
Halicioğlu Data Science Institute
UC San Diego

- Research Interests:
 - Promoting healthy study strategies at scale by
 - using AI-powered educational tools for better student metacognition & instructor/student interaction
 - psychotherapy techniques on a mobile application (willowmind.io)
- Teaching:
 - Data structures and algorithms
 - Introduction to computer science
- Fun: hiking/camping, k-dramas, parenting two kids (7 and 3)

Let's create the name tags so I can remember your name



Natural Language Processing

Discuss:

1. What do you think Natural Language Processing (NLP) is?
2. What are some applications of NLP?

AI works with many different types of data

- tables
- images
- sounds
- texts

NLP deals with text data

1. text analysis

- a. analyze given text and derive any insight

2. text generation

- a. auto-completion, auto-correction

3. And we are going to work on a simplest NLP application, called sentiment analysis

Sentiment analysis determines whether a given text is positive or negative

Given a movie review,

“it was a terrific movie” -> positive

“it was an awful movie” -> negative

The analysis follow a machine learning model, called Naive Bayes classifier

Naive Bayes Classifier

Following is a dataset of 1000 fruits. The dataset consists of four columns. The first column (Fruit Name) has three unique values - Orange, Banana, and Other. The rest three columns are binary data.

Fruit	Long	Sweet	Yellow
Orange	0	1	0
Banana	1	0	1
Banana	1	1	1
Other	1	1	0
..	..	..	..

Answer:

What is $P(\text{Fruit is a Banana})$?

What is $P(\text{Fruit is a Long Banana})$?

What is $P(\text{Fruit} = \text{Banana} \mid \text{Long} = 1)$?

What is $P(\text{Sweet} = 1 \mid \text{Fruit} = \text{Banana})$?

Summary of the raw dataset

Fruit	Long	Not Long	Sweet	Not Sweet	Yellow	Not Yellow	Total
Banana	400	100	350	150	450	50	500
Orange	0	300	150	150	300	0	300
Other	100	100	150	50	50	150	200
Total	500	500	650	350	800	200	1000

Naive Bayes classifier is based on “Bayes rule”

and Bayes rule is all about conditional probability!

$$P(Y|X) = \frac{P(X|Y) * P(Y)}{P(X)}$$

And there are names for three terms

and Bayes rule is all about conditional probability!

$$\text{Posterior Probability } P(Y|X) = \frac{\text{Likelihood of Evidence } P(X|Y) * \text{Prior Probability } P(Y)}{P(X)}$$

Following is a dataset of 1000 fruits. The dataset consists of four columns. The first column (Fruit Name) has three unique values - Orange, Banana, and Other. The rest three columns are binary data.

Fruit	Long	Not Long	Sweet	Not Sweet	Yellow	Not Yellow	Total
Banana	400	100	350	150	450	50	500
Orange	0	300	150	150	300	0	300
Other	100	100	150	50	50	150	200
Total	500	500	650	350	800	200	1000

Question:

You are given a fruit that is long, sweet, and yellow. What is that fruit the most likely to be?

Answer: The fruit which has the maximum of following probabilities!

$P(\text{Fruit} = \text{Banana} \mid (\text{Long}=1, \text{Sweet}=1, \text{and Yellow}=1))$

$P(\text{Fruit} = \text{Orange} \mid (\text{Long}=1, \text{Sweet}=1, \text{and Yellow}=1))$

$P(\text{Fruit} = \text{Other} \mid (\text{Long}=1, \text{Sweet}=1, \text{and Yellow}=1))$

Answer: The fruit which has the maximum of following probabilities!

$P(\text{Fruit} = \text{Banana} \mid (\text{Long}=1, \text{Sweet}=1, \text{and Yellow}=1))$

$P(\text{Fruit} = \text{Orange} \mid (\text{Long}=1, \text{Sweet}=1, \text{and Yellow}=1))$

$P(\text{Fruit} = \text{Other} \mid (\text{Long}=1, \text{Sweet}=1, \text{and Yellow}=1))$

$$P(Y|X) = \frac{P(X|Y) * P(Y)}{P(X)}$$

Y: Fruit = Banana

X:

1) Long = 1

2) Sweet = 1

3) Yellow = 1

“Naive” Bayes assumes no dependence across input features

1. Meaning there is no correlation between a fruit being long, sweet, and yellow (= they are all independent events)
2. This allows us to calculate a joint probability easily
 - a. $P(\text{Long and Banana}) = P(\text{Long}) * P(\text{Banana})$

Fruit	Long	Not Long	Sweet	Not Sweet	Yellow	Not Yellow	Total
Banana	400	100	350	150	450	50	500
Orange	0	300	150	150	300	0	300
Other	100	100	150	50	50	150	200
Total	500	500	650	350	800	200	1000

$P(\text{Fruit} = \text{Banana} \mid (\text{Long}=1 \text{ and Sweet}=1 \text{ and Yellow}=1))$

$$P(Y|X) = \frac{P(X|Y) \cdot P(Y)}{P(X)}$$

$Y: \text{Fruit} = \text{Banana}$

$X: \text{Long} = 1 \text{ and Sweet} = 1 \text{ and Yellow} = 1$

$$P(Y) = P(\text{Banana}) = 500/1000 = 0.5$$

Fruit	Long	Not Long	Sweet	Not Sweet	Yellow	Not Yellow	Total
Banana	400	100	350	150	450	50	500
Orange	0	300	150	150	300	0	300
Other	100	100	150	50	50	150	200
Total	500	500	650	350	800	200	1000

$P(\text{Fruit} = \text{Banana} \mid (\text{Long}=1 \text{ and Sweet}=1 \text{ and Yellow}=1))$

$$P(Y|X) = \frac{P(X|Y) * 0.5}{P(X)}$$

$Y: \text{Fruit} = \text{Banana}$

$X: \text{Long} = 1 \text{ and Sweet} = 1 \text{ and Yellow} = 1$

$$P(X|Y) = P(\text{Long} \mid \text{Banana}) * P(\text{Sweet} \mid \text{Banana}) * P(\text{Yellow} \mid \text{Banana})$$

$$P(\text{Long} \mid \text{Banana}) = 400/500 = 0.8$$

$$P(\text{Sweet} \mid \text{Banana}) = 350/500 = 0.7$$

$$P(\text{Yellow} \mid \text{Banana}) = 450/500 = 0.9$$

$$\begin{aligned} P(X|Y) \\ &= 0.8 * 0.7 * 0.9 \\ &= \mathbf{0.504} \end{aligned}$$

Fruit	Long	Not Long	Sweet	Not Sweet	Yellow	Not Yellow	Total
Banana	400	100	350	150	450	50	500
Orange	0	300	150	150	300	0	300
Other	100	100	150	50	50	150	200
Total	500	500	650	350	800	200	1000

$P(\text{Fruit} = \text{Banana} \mid (\text{Long}=1 \text{ and Sweet}=1 \text{ and Yellow}=1))$

$$P(Y|X) = \frac{0.504 * 0.5}{P(X)}$$

Y: Fruit = Banana

*X: Long = 1 **and** Sweet = 1 **and** Yellow = 1*

$$P(X) = P(\text{Long} \cap \text{Sweet} \cap \text{Yellow}) = P(\text{Long}) * P(\text{Sweet}) * P(\text{Yellow})$$

$$= 0.5 * 0.65 * 0.8 = 0.26$$

Fruit	Long	Not Long	Sweet	Not Sweet	Yellow	Not Yellow	Total
Banana	400	100	350	150	450	50	500
Orange	0	300	150	150	300	0	300
Other	100	100	150	50	50	150	200
Total	500	500	650	350	800	200	1000

$P(\text{Fruit} = \text{Banana} \mid (\text{Long}=1 \text{ and Sweet}=1 \text{ and Yellow}=1))$

$$P(Y|X) = \frac{0.504 * 0.5}{0.26} = 0.969$$

Y: Fruit = Banana

X: Long = 1 and Sweet = 1 and Yellow = 1

Fruit	Long	Not Long	Sweet	Not Sweet	Yellow	Not Yellow	Total
Banana	400	100	350	150	450	50	500
Orange	0	300	150	150	300	0	300
Other	100	100	150	50	50	150	200
Total	500	500	650	350	800	200	1000

$$P(Banana|X) = \frac{0.504 * 0.5}{0.26} = 0.969$$

When the fruit is long, sweet, and yellow, it's likely to be a banana!

$$P(Orange|X) = \frac{0 * 0.3}{0.26} = 0$$

$$P(Other|X) = \frac{0.0375 * 0.5}{0.26} = 0.072$$

Consideration #1 about the Bayes rule

$P(X)$ stays constant as 0.26, when calculating three different posterior probability

a. so we don't calculate it ..:D

$$P(Y|X) = \frac{P(X|Y) * P(Y)}{P(X)}$$

$$P(Banana|X) = \frac{0.504 * 0.5}{0.26} = 0.969$$

$$P(Orange|X) = \frac{0 * 0.3}{0.26} = 0$$

$$P(Other|X) = \frac{0.0375 * 0.5}{0.26} = 0.072$$

Consideration #2 about the Bayes rule

2. $P(\text{Orange}|X)$ becomes 0 simply because of a single term $P(\text{Long}|\text{Orange})$

a. too extreme. so we lessen its impact with smoothing

$$P(Y|X) = \frac{P(X|Y) * P(Y)}{P(X)}$$

$$P(\text{Banana}|X) = \frac{0.504 * 0.5}{0.26} = 0.969$$

$$P(\text{Orange}|X) = \frac{0 * 0.3}{0.26} = 0$$

$$P(\text{Other}|X) = \frac{0.0375 * 0.5}{0.26} = 0.072$$

Laplace smoothing lessons the impact of a single 0 term

add 1 to denominator and # of input features to the numerator, when calculating the likelihood of evidence

$$P(Y|X) = \frac{P(X|Y) * P(Y)}{P(X)}$$

$$P(X|Y) = P(\text{Long} | \text{Orange}) * P(\text{Sweet} | \text{Orange}) * P(\text{Yellow} | \text{Orange})$$
$$= (0+1)/(300+3) = 0.003$$

Naive Bayes Classifier with Text Data

Text	Sentiment
just plain boring	neg
entirely predictable and lacks energy	neg
no surprises and very few laughs	neg
very powerful	pos
the most fun film of the summer	pos

Predict the sentiment of a text T: **“predictable with no fun”**

$$P(Y|X) = \frac{P(X|Y) * P(Y)}{P(X)}$$

Y: Sentiment = neg or pos

*X: each word in T (predictable, with, no, fun)
and we assume each word occurs
independently*

Text	Sentiment
just plain boring	neg
entirely predictable and lacks energy	neg
no surprises and very few laughs	neg
very powerful	pos
the most fun film of the summer	pos

Predict the sentiment of a text T:
“predictable with no fun”

$$P(Y|X) = \frac{P(X|Y) * P(Y)}{P(X)}$$

ignore P(X), since it's a constant

Y: Sentiment = neg or pos

*X: each word in T (predictable, with, no, fun)
 and we assume each word occurs
 independently*

Text	Sentiment
just plain boring	neg
entirely predictable and lacks energy	neg
no surprises and very few laughs	neg
very powerful	pos
the most fun film of the summer	pos

Predict the sentiment of a text T:
“predictable with no fun”

$$P(Y|X) = P(X|Y) * P(Y)$$

Prior probability

*Y: Sentiment = **pos***

*X: each word in T (predictable, with, no, fun)
and we assume each word occurs
independently*

$$P(Y) = P(pos) = 2/5$$

Text	Sentiment
just plain boring	neg
entirely predictable and lacks energy	neg
no surprises and very few laughs	neg
very powerful	pos
the most fun film of the summer	pos

Predict the sentiment of a text T:
“predictable with no fun”

$$P(Y|X) = P(X|Y) * 0.4$$

*Y: Sentiment = **pos***

*X: each word in T (predictable, with, no, fun)
 and we assume each word occurs
 independently*

$P(X|Y)$ with Laplace smoothing applied

$$= P(\text{word}_{\text{predictable}} | \text{pos}) * P(\text{word}_{\text{with}} | \text{pos}) * P(\text{word}_{\text{no}} | \text{pos}) * P(\text{word}_{\text{fun}} | \text{pos})$$

$$= \frac{0+1}{9+20} * \frac{0+1}{9+20} * \frac{1+1}{9+20} = \frac{1 * 1 * 2}{29^3}$$

**ignore this, since with doesn't exist
 in the training dataset**

Text	Sentiment
just plain boring	neg
entirely predictable and lacks energy	neg
no surprises and very few laughs	neg
very powerful	pos
the most fun film of the summer	pos

Predict the sentiment of a text T:
“predictable with no fun”

$$P(Y|X) = P(X|Y) * P(Y)$$

$$P(Y = pos | X) = \frac{0 + 1}{9 + 20} * \frac{0 + 1}{9 + 20} * \frac{1 + 1}{9 + 20} * 0.4 = 3.2 * 10^{-5}$$

$$P(Y = neg | X) = \frac{1 + 1}{14 + 20} * \frac{1 + 1}{14 + 20} * \frac{0 + 1}{14 + 20} * 0.6 = 6.1 * 10^{-5}$$

Naive Bayes Classifier Implementation

Step I: Split texts in the training dataset into positive and negative groups

Text	Sentiment
just plain boring	neg
entirely predictable and lacks energy	neg
no surprises and very few laughs	neg
very powerful	pos
the most fun film of the summer	pos



Negative Only

Text
just plain boring
entirely predictable and lacks energy
no surprises and very few laughs

Positive Only

Text
very powerful
the most fun film of the summer

Step2: Calculate Prior probability

Negative Only

Text
just plain boring
entirely predictable and lacks energy
no surprises and very few laughs



Prior Probability

pos: 2/5

neg: 3/5

Positive Only

Text
very powerful
the most fun film of the summer



Step3: Count occurrences for each word, for each sentiment class

Negative Only

Text

just plain boring
entirely predictable and
lacks energy
no surprises and very
few laughs



just	1
plain	1
boring	1
entirely	1
predictable	1
and	2
lacks	1
energy	1
no	1
surprises	1
very	1
few	1
laughs	1

called “bag of words”
notation

Positive Only

Text

very powerful
the most fun film of the
summer

Step3: Count occurrences for each word, for each sentiment class

Negative Only

Text
just plain boring
entirely predictable and lacks energy
no surprises and very few laughs

Positive Only

Text
very powerful
the most fun film of the summer



very	1
powerful	1
the	1
most	1
fun	1
film	1
of	1
the	1
summer	1

For Step3, you need to break down each review text into words

1. tokenizing

- a. convert a text “just plain boring” to a list of words ['just', 'plain', 'boring']
- b. you will need to do some pre-processing

2. remove symbols

- a. remove extra spaces
- b. remove some strange characters

3. And more! (extra)

For Step3, you also need to count the number of total unique words

across pos/neg class, so that you know what number to add for the Laplace smoothing

Negative
Only

Text
just plain boring
entirely predictable and lacks energy
no surprises and very few laughs

Positive
Only

Text
very powerful
the most fun film of the summer

There are total 20 unique words:

just, plain, boring, entirely, predictable, and, lacks, energy, no, surprises, very, few, laughs, powerful, the, most, fun, film, of, summer

Step4: Calculate posterior probability, for each class, in a log form

I. the posterior probability is often very small

a. use `math.log()`, so that the program can handle small numbers

$$P(Y|X) = P(X|Y) * P(Y)$$

$$P(Y = pos | X) = \frac{0+1}{9+20} * \frac{0+1}{9+20} * \frac{1+1}{9+20} * 0.4 = 3.2 * 10^{-5}$$



You can get this information by looking at bag-of-words

$$\log(P(Y = pos | X)) = \log\left(\frac{0+1}{9+20}\right) + \log\left(\frac{0+1}{9+20}\right) + \log\left(\frac{1+1}{9+20}\right) + \log(0.4)$$

$$= \log(3.2 * 10^{-5})$$

Step5: Choose max of the two posterior probabilities

- I. if $\log(P(Y = pos|X))$ is larger than $\log(P(Y = neg|X))$, the text is likely to have positive sentiment; otherwise negative

Activity I

Write a Python function to split the input text (represented as a string) into individual sentences (a list of strings).

```
def get_sentences(text):  
    # TODO: define your function here
```

Sample input: “Cluster I2 is awesome. I love COSMOS.”

Sample output: [“Cluster I2 is awesome”, “I love COSMOS”]

Hint: Use Inbuilt Python Functions

What does your program do with the following input?

“Dr. Sahay and Prof. Soosairaj are the instructors for Cluster 12.”

Backup

training logistic regression

- With linear regression, we can derive a closed-form solution for the parameters in terms of the least-squares equations

- For logistic regression, let's consider the **likelihood** of the model over data samples $i = 1, \dots, n$:

$$L(\beta) = \prod_{i=1}^n p(y_i|x_i, \beta) = \prod_{i=1}^n (f_{\beta}(x_i))^{y_i} \cdot (1 - f_{\beta}(x_i))^{1-y_i}$$

when $y_i = 1$, we want to maximize $f_{\beta}(x_i)$, and
when $y_i = 0$, we want to maximize $1 - f_{\beta}(x_i)$

- And then the **log likelihood**, which is easier to optimize

$$\begin{aligned} l(\beta) &= \sum_{i=1}^n \log[(f_{\beta}(x_i))^{y_i} \cdot (1 - f_{\beta}(x_i))^{1-y_i}] \\ &= \sum_{i=1}^n [y_i \log f_{\beta}(x_i) + (1 - y_i) \log(1 - f_{\beta}(x_i))] \end{aligned}$$

- There is no (known) closed form solution to maximize $l(\beta)$, given the $\log f_{\beta}(x_i)$ terms

gradient descent (ascent)

- We want to find β to *maximize* $l(\beta)$ but no closed-form exists.
- Consider the **gradient descent (ascent)** algorithm, an iterative procedure for finding a **local minimum (maximum)** of a function by moving away from (towards) the gradient:

$$\beta_j^{t+1} = \beta_j^t - \alpha^t \frac{\partial}{\partial \beta_j} l(\beta^t), \quad \beta_j^{t+1} = \beta_j^t + \alpha^t \frac{\partial}{\partial \beta_j} l(\beta^t)$$

Gradient Descent Gradient Ascent

- Here, α^t is the **step size** of the algorithm at time t
- Since $l(\beta)$ is a **concave** function, we can *guarantee* that gradient ascent will eventually converge to the **global maximum**, so long as certain conditions on α^t are met

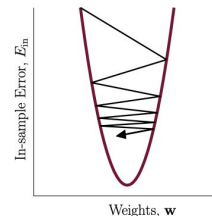
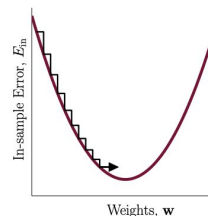
for non-convex functions,
no guarantee of
convergence to optimum



in general, the step size
must be tuned correctly

η too small

η too large



gradient ascent example

Suppose we have a single parameter b for some model we are trying to train. For this model, we find a log-likelihood function of

$$l(b) = -\left(\frac{b - m}{s}\right)^2$$

where m and s are constants. Derive the iterative procedure for determining the model parameters as a function of the step size α^t . Run the procedure for different values of α^t until $t = 10$ and compare the results.

solution

$$l(b) = -\left(\frac{b-m}{s}\right)^2$$

We always want to maximize the log-likelihood, so we use gradient ascent.
Letting b^t be the value of b at iteration t , our update procedure will be:

$$b^{t+1} = b^t + \alpha^t \frac{d}{db} l(b^t)$$

Evaluating the derivative, this becomes:

$$b^{t+1} = b^t - 2\alpha^t \left(\frac{b^t - m}{s^2}\right)$$

Suppose $\alpha = 0.1, m = 5, s = 0.7$.
If we start at $b^0 = 1.1$ (arbitrary), we get

$$\begin{aligned} b^1 &= 1.1 - 2 \cdot 0.1 \cdot \left(\frac{1.1 - 5}{0.7^2}\right) \\ &= 2.692 \end{aligned}$$

$$\begin{aligned} b^2 &= 2.692 - 2 \cdot 0.1 \cdot \left(\frac{2.692 - 5}{0.7^2}\right) \\ &= 3.634 \end{aligned}$$

...

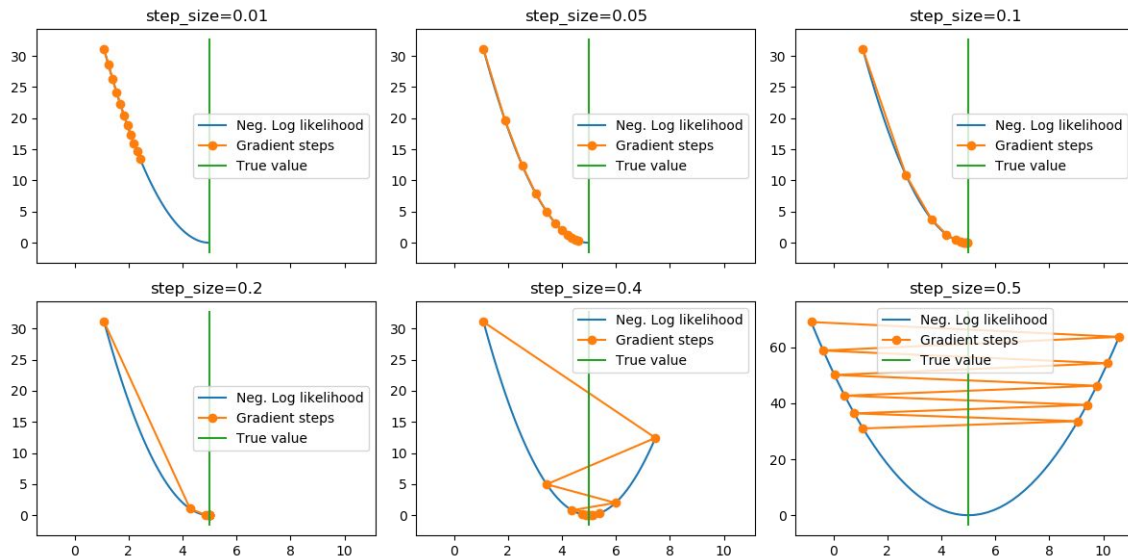
$$\begin{aligned} b^{10} &= 4.965 - 2 \cdot 0.1 \cdot \left(\frac{4.965 - 5}{0.7^2}\right) \\ &= 4.979 \end{aligned}$$

solution

Below, we plot the evolution of b^t over t (see the Jupyter notebook), starting with $b^0 = 1.1$ for $\alpha^t = 0.01, 0.05, 0.1, 0.2, 0.4, 0.5$. Again, we set $m = 5$ and $s =$

Here, the y-axis is actually $-l(b)$, to make the values positive. Maximizing the log-likelihood is equivalent to minimizing the negative log-likelihood.

Tuning α^t is a very important question!



gradient ascent for logistic regression

variable η_t – just right

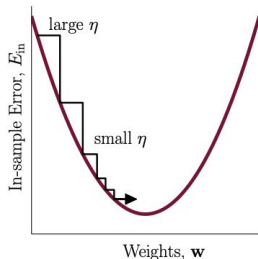
- Back to logistic regression. Evaluating the partial derivative,

$$\begin{aligned} \frac{\partial}{\partial \beta_j} l(\beta) &= \frac{\partial}{\partial \beta_j} \sum_{i=1}^n [y_i \log f_\beta(x_i) + (1 - y_i) \log(1 - f_\beta(x_i))] \\ &= \sum_{i=1}^n \left(\frac{y_i}{f_\beta(x_i)} - \frac{1 - y_i}{1 - f_\beta(x_i)} \right) \frac{\partial}{\partial \beta_j} f_\beta(x_i) \quad \begin{array}{l} \text{Partial derivative of loss with respect to } f_\beta(x_i) \\ \text{Partial derivative of logistic function } g(v) \\ \text{with respect to } v_i \equiv \beta_1 x_{i1} + \beta_2 x_{i2} + \dots \end{array} \\ &= \sum_{i=1}^n \left(\frac{y_i}{f_\beta(x_i)} - \frac{1 - y_i}{1 - f_\beta(x_i)} \right) f_\beta(x_i) (1 - f_\beta(x_i)) \frac{\partial}{\partial \beta_j} (\dots + \beta_j x_{ij} + \dots) \\ &= \sum_{i=1}^n (y_i (1 - f_\beta(x_i)) - (1 - y_i) f_\beta(x_i)) x_{ij} = \sum_{i=1}^m (y_i - f_\beta(x_i)) x_{ij} \quad \begin{array}{l} \text{Partial derivative of } v_i \\ \text{with respect to } \beta_j \end{array} \end{aligned}$$

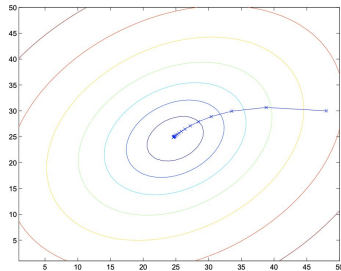
- Thus, we get the following gradient ascent rule for logistic regression:

$$\beta_j^{t+1} = \beta_j^t + \alpha^t \left[\sum_{i=1}^n (y_i - f_\beta(x_i)) x_{ij} \right], j \geq 1$$

$$\beta_0^{t+1} = \beta_0^t + \alpha^t \left[\sum_{i=1}^n (y_i - f_\beta(x_i)) \right]$$



$$\begin{aligned} \frac{\partial}{\partial v} g(v) &= \frac{\partial}{\partial v} (1 + e^{-v})^{-1} = -(1 + e^{-v})^{-2} e^{-v} (-1) \\ &= a(v)(1 - a(v)) \end{aligned}$$



Evaluation Metric

Discuss:

A brand-new COVID testing kit recorded 89% accuracy. Is this good?

Discuss:

Here's the breakdowns. Is this still good?
Why or why not?

		Ground Truth	
		Positive	Negative
Test Result	Positive	5	1
	Negative	10	84

binary evaluation metrics

see `sklearn.metrics`
in Python

- In classification, our predictions are either right or wrong
- For binary classification, we typically report several metrics (on a test set), based on a **confusion matrix** (shown to the right). The most common three are:

		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$

- **Accuracy**: Fraction of correct predictions
- **Precision**: Fraction of correct predictions in the predicted positive class
- **Recall** (or **sensitivity**): Fraction of correct predictions in the actual positive class

Discuss:

Come up with an example scenario where a specific metric is more important than the others

composite binary metrics

- With classification problems, none of these confusion table metrics tell the whole story:
 - If there is significant **class imbalance**, accuracy can look very good even if the classifier is not
 - For example, suppose 90% of cars are minivans and 10% are sports cars. If we always predict minivan, we will have 90% accuracy!
- There are two composite metrics that can be useful:
 - **F1 score**: Harmonic mean between precision and recall (both need to be high for the F1 score to be high) = $2 * p * r / (p + r)$
 - **AUROC**: Area under true/false positive curve from varying decision threshold from 0 (predict all negatives) and 1 (predict all positives)

